

Module 22: Integration

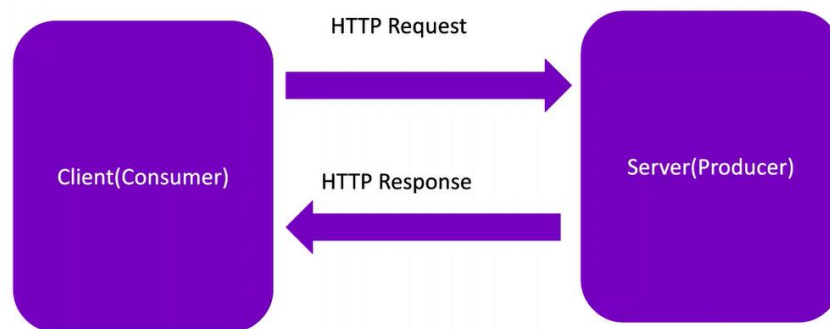
Apex Web Services

Integration Overview

- Salesforce integration define process of connecting salesforce platform with other application.
- Every application has presentation Layer, Security Layer and business Layer.
- The purpose of integration is sharing data or automate business processing from one system to another.
- Integration provide data accuracy as well as consistency.
- You can utilize numerous way to integrate data in salesforce platform such as change data Capture, Custom Apex(Rest and Soap Web Services), Salesforce connect/ External Object, Omni studio Integration Procedure or Native Salesforce APIS.

Web Services

Web services is way to exchange data from one application to another application over world wide web.



Web Services (continue ..)

- Sender of data from one system to another system over internet using web services is called **Producer**.
- Receiver of data in web service is called **Consumer**.
- Design a mechanism for sharing information using apex on salesforce platform, we utilize SOAP API or REST API technique.
- To call data from distinct application using apex REST API or SOAP API in salesforce platform, is called "callout web services".
- To share data from salesforce platform to another application using apex REST API or SOAP API, is called "expose web services".

SOAP Webservices

- SOAP Stands for Simple Object Access Protocols.
- XML-based protocol for accesses web services.
- Support XML format data only.

Difference Between Enterprise WSDL And Partner WSDL

Enterprise WSDL	Partner WSDL
Enterprise WSDL is strongly typed, fields and objects with different data types—like int and string—are included.	The Partner WSDL uses name-value pairs of field names and values instead of specific data types.
Contains the metadata about all standard and custom fields and objects.	Does not contain metadata about objects and fields. Takes an array of key-value pairs.
It is used for single organization solution.	It is used for Multiple Org Solution.

Expose Apex Method As SOAP Web Services

- Define a global class .
- Use webservice keyword to define web service method.

```
global class MyWebService {  
  
    webservice static Return Type method()  
    {  
  
    }  
  
}
```

Integration: connecting the salesforce org with any external application is integration. There are different tools available for this.

1. Salesforce connect- no code is required.
2. Apex Code-SOAP API and REST API
3. Change data capture --> which is MuleSoft tool.

SOAP API

1. Exposing the Web service
2. Consuming the web service

REST API

1. Exposing the Web service
2. Consuming the web service

SOAP API

1. Exposing the Web service: You have some code written in Apex class and you want that functionality to be used by some other external application so you can expose that Apex class.

Expose Apex Method As SOAP Web Services

Developed a SOAP API for creating a new position.

```
global class PositionWebService {
    webservice static String savePosition (String positionName,String jobdescription)
    {
        Position__c pos=new Position__c();
        pos.Name=positionName;
        pos.Job_Description__c=jobdescription;
        insert pos;
        return pos.id;
    }
}
```

REST Webservices

- REST Stands for Representational State Transfer .
- It is an architecture design to construct web services using HTTP.
- It is simple & robust web service based on RESTful principle.
- It supports multi formats data, however data is often in formats like JSON or XML in salesforce platform.

Difference Between SOAP And REST Webservices

SOAP	REST
SOAP is protocol.	REST is an architecture.
Only allows XML data format.	Allows XML and JSON data format.
Due to their larger size and increased complexity, SOAP messages transmit and process more slowly, potentially leading to longer page load times.	Compared to SOAP, REST is faster and more efficient because of its smaller message sizes.
Scaling SOAP can be challenging due to its stateful nature, where the server stores all previous messages exchanged with a client.	Scaling REST is simple. Because it is stateless, each communication is handled separately from the ones that came before it.
Useful in legacy applications .	useful in modern/ mobile applications .

Request Structure for REST API

Request Items	Description
URI(Uniform Resource Identifier)	It is used to identify the resources. Every resource is accessed via URI. URI structure for salesforce https://MyDomainName.my.salesforce.com/services/data/vXX.X/resource/
HTTP Methods	You can use various standard HTTP method to interact with resource. These are GET,POST,PUT, Delete and Patch.
Request Body	Provide request body for update or insert operation on resource. Request body can JSON or XML format based on requested content type.
Request Header	It is used to provide additional informational about request being made. Request Headers are key value pairs such as authorization: value, content type : value , and accept-language :value etc.

Response Structure of REST

Every REST response contains two parts:

- Response Body
- Response Status Code

Expose REST API In Salesforce

Some important annotation are used to design REST API as expose service.

Annotation	Description	Action
@RestResource(urlMapping="/url")	Define a class with this annotation & specify resource endpoint using urlMapping attribute.	NA
@HttpGet	Used to fetch record from database.	Read
@HttpPost	Used to create record in database.	Insert
@HttpPut	Used to update record in database	Update
@HttpDelete	Used to delete record in database	Delete
@HttpPatch	Used to partial update record in database.	Upsert

Expose a Class as a REST Service

- Define a class with global Access Modifier
- Add `@RestResource(urlMapping='/endpoint/')` annotation statement above the class statement
- Methods that will be exposed as endpoint, Define as global & static with HTTP method annotation based on functionality.

↳

```
@RestResource(urlMapping='/endpoint/')
public class MyObjectRestResource {
    @HttpPost
    global static ReturnType doPost()
    {
        // Business logic for POST Request
    }
    @HttpGet
    global static ReturnType doGet()
    {
        // Business logic for GET Request
    }
    @HttpPut
    global static ReturnType doPut()
    {
        // Business logic for PUT Request
    }
    @HttpDelete
    global static ReturnType doDelete()
    {
        // Business logic for DELETE Request
    }
}
```

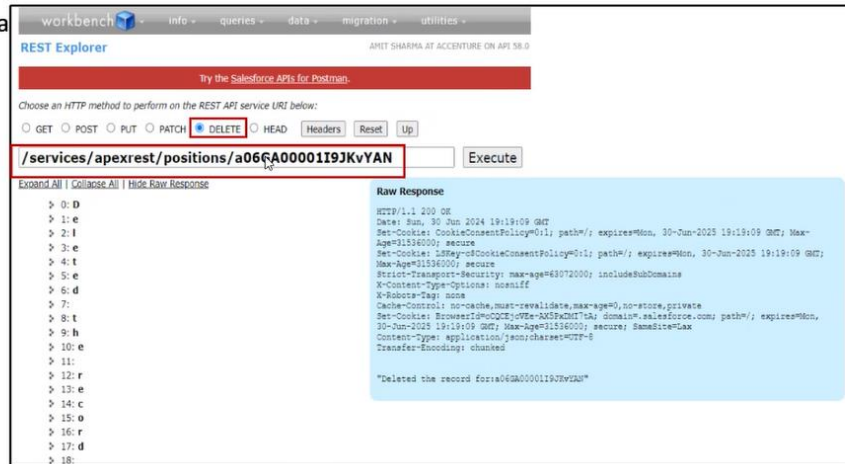
Expose Rest API In Salesforce(Continue ..)

Designed class 'PositionRestSource' to create position record and delete a position record with specified record Id :

```
@RestResource(urlMapping='/positions/*')
global class PositionRestResource
{
    @HttpPost
    global static String doPost(String name, String jobDescription, String status, String department )
    {
        Position__c pos=new Position__c();
        pos.Name=name;
        pos.Job_Description__c=jobDescription;
        pos.Status__c=status;
        pos.Department__c=department;
        try{
            insert pos;
            return pos.Id;
        }catch(DMLException e)
        {
            return e.getMessage();
        }
    }
}
```


Verify RestResource Using Workbench

Verify delete record functional
PositionRestResource



Project Name – Car Rental Application.

A Car Rental App helps the customers to book a car for rent and pay according to the usage of the car. This service is available across multiple cities and they want an application in Salesforce to maintain this data.

Design an application with below given requirements:

1. Create a data model with object and field names as below: There are 3 Objects – Customer, Car and Rental. (Rental is a Junction Object)

Customer Fields and its data types:

Name → Text, Age → number, Contact → Phone, Email → Email, Address → Text, driving license number → Text, rental start date → Date, rental end date → Date

Car Fields and its data types:

Name → Text, model → Text, current condition of the car → Text, current fuel level → number, current miles/km → number, Car Availability → Picklist. Picklist values are – (Available, Not available), location → text, Rental cost → number, category → Picklist. Picklist values are – (Mini, Premium, luxury).

Rental:

Customer Name → (Master-Detail(Customer)), Car Name → Master-Detail(Car)), Total number of days (for which Car is on rent) → Formula Field, Total Rental Cost → Formula Field, Car Return

1. Create a data model with object and field names as below: There are 3 Objects – Customer, Car and Rental. (Rental is a Junction Object)

Customer Fields and its data types:

Name → Text, Age → number, Contact → Phone, Email → Email, Address → Text, driving license number → Text, rental start date → Date, rental end date → Date

Car Fields and its data types:

Name → Text, model → Text, current condition of the car → Text, current fuel level → number, current miles/km → number, Car Availability → Picklist. Picklist values are – (Available, Not available), location → text, Rental cost → number, category → Picklist. Picklist values are – (Mini, Premium, luxury).

Rental:

Customer Name → (Master-Detail(Customer)), Car Name → Master-Detail(Car)), Total number of days(for which Car is on rent) → Formula Field, Total Rental Cost → Formula Field, Car Return Status → Picklist(Picklist Values are – Returned, Not Returned)

2. Create a formula field in Rental Object as Below:
 - a) Total number of days(for which Car is on rent) - To calculate the total number of days the car is taken on rent.
 - b) Total Rental Cost - Create a formula field to calculate the total cost of a Rental car.
3. Create a flow to automatically update the Car Availability to 'Not Available' if the car is already rented to any customer. (Hint: Check in the Rental Object if the Customer and Car Field is not NULL).

4. Create a Summary report which should display Cars by its Category.

1. Create a data model with object and field names as below: There are 3 Objects – Customer, Car and Rental. (Rental is a Junction Object)

Customer Fields and its data types:

Name → Text, Age → number, Contact → Phone, Email → Email, Address → Text, driving license number → Text, rental start date → Date, rental end date → Date

Car Fields and its data types:

Name → Text, model → Text, current condition of the car → Text, current fuel level → number, current miles/km → number, Car Availability → Picklist. Picklist values are – (Available, Not available), location → text, Rental cost → number, category → Picklist. Picklist values are – (Mini, Premium, luxury).

Rental:

Customer Name → (Master-Detail(Customer)), Car Name → Master-Detail(Car), Total number of days (for which Car is on rent) → Formula Field, Total Rental Cost → Formula Field, Car Return Status → Picklist (Picklist Values are – Returned, Not Returned)

2. Create a formula field in Rental Object as Below:
 - a) Total number of days (for which Car is on rent) - To calculate the total number of days the car is taken on rent.
 - b) Total Rental Cost - Create a formula field to calculate the total cost of a Rental car.
3. Create a flow to automatically update the Car Availability to 'Not Available' if the car is already rented to any customer. (Hint: Check in the Rental Object if the Customer and Car Field is not NULL).

4. Create a Summary report which should display Cars by its Category.

2. Create a formula field in Rental Object as Below:
 - a) Total number of days(for which Car is on rent) - To calculate the total number of days the car is taken on rent.
 - b) Total Rental Cost - Create a formula field to calculate the total cost of a Rental car.
3. Create a flow to automatically update the Car Availability to 'Not Available' if the car is already rented to any customer. (Hint: Check in the Rental Object if the Customer and Car Field is not NULL).
4. Create a Summary report which should display Cars by its Category.
5. Write a Parent to Child SOQL query to get the Cars and Rentals related to those cars. (Hint: Car is a Parent Object and Rental is a Child object)
6. Create a Validation Rule to enter the driving license number in proper format. (Use Regular Expressions)

7. Write a trigger to Change the Car Availability Status to 'Available', once the Car is returned. (Hint: Use the Car Return Status Field on Rental Object)
8. Write a LWC Component to display the Car Records only for the Cars available for Rent. (Use Imperative Approach)

-
3. Create a flow to automatically update the Car Availability to 'Not Available' if the Car is already rented to any customer. (Hint: Check in the Rental Object if the Customer and Car Field is not NULL).
 4. Create a Summary report which should display Cars by its Category.
 5. Write a Parent to Child SOQL query to get the Cars and Rentals related to those cars. (Hint: Car is a Parent Object and Rental is a Child object)
 6. Create a Validation Rule to enter the driving license number in proper format. (Use Regular Expressions)

I

-
7. Write a trigger to Change the Car Availability Status to 'Available', once the Car is returned. (Hint: Use the Car Return Status Field on Rental Object)
 8. Write a LWC Component to display the Car Records only for the Cars available for Rent. (Use Imperative Approach)
-